

NODE.js KNOWLEDGE BYTES

- **Backend:** Node.js 22 + Express 4 + pg
- **Database:** Same PostgreSQL placement_tracker database, but using the separate nodejs_applications table.
- **Frontend:** Existing Vue frontend was kept unchanged and verified byte-identical.
- **API:** CRUD endpoints verified directly on :3000 and through NGINX on :8080.
- **Tests: 43/43 passed.**
- **Original Applications table:** Confirmed untouched and still contains the original 3 records.
- **Docker:** Exactly **one** service/container, app.
- **NGINX:** Vue served on :8080, /api/ proxied to Node.js :3000.
- **Persistence:** Verified across Docker restart.
- **Data migration:** The 3 existing records were copied into nodejs_applications, with IDs preserved and no duplication on subsequent initialization.

node_modules is the folder where **Node.js project dependencies/packages are installed.**

In your project, packages like:

- express → backend web framework
- pg → PostgreSQL connection
- cors → handles cross-origin requests
- supertest → API testing

are installed inside node_modules.

In simple words:

package.json says which packages the project needs, and **node_modules** contains the actual installed packages/code.

You usually create it by running:

```
npm install
```

You **don't manually edit node_modules.**

init-db.js

Byte 1: Purpose of the Database Initialization Script

Builds on: None — starting point

In plain terms:

This script prepares the PostgreSQL database specifically for the Node.js version of the Placement Tracker. It creates the Node-specific table if needed and copies existing application data only when that table is empty.

The code:

```
// 1. CREATE TABLE IF NOT EXISTS nodejs_applications
// 2. Copy legacy "Applications" data if the new table is empty
// 3. Reset the ID sequence
```

What's happening:

The script is designed to be **idempotent**, meaning it can safely run multiple times without repeatedly creating or duplicating data.

Why it matters:

Docker may run this script every time the container starts. Idempotency prevents every restart from creating duplicate records.

Byte 2: Importing the Database Dependencies

Builds on: Byte 1

In plain terms:

The script needs two things: a PostgreSQL connection pool and a function that makes sure the required table exists.

The code:

```
const pool = require('./src/db/pool');
const { ensureSchema } = require('./src/db/schema');
```

What's happening:

pool provides database connections. ensureSchema handles creating nodejs_applications if it does not already exist.

Why it matters:

This keeps database connection logic and table-definition logic separate from the initialization workflow.

Byte 3: Checking Whether a Table Exists

Builds on: Byte 2

In plain terms:

Before trying to copy data from the old Applications table, the script checks whether that table actually exists.

The code:

```
async function tableExists(client, name) {  
  const result = await client.query(  
    'SELECT to_regclass($1) AS oid',  
    [name]  
  );  
  return result.rows[0].oid !== null;  
}
```

What's happening:

PostgreSQL's `to_regclass()` returns the table's identifier when the table exists. If it doesn't exist, the result is null.

Why it matters:

A fresh database might not contain the old .NET Applications table. This check allows the script to handle that situation safely.

Byte 4: Getting a Database Connection**Builds on:** Byte 2**In plain terms:**

`initDb()` is the main initialization function. It first obtains a database connection from the connection pool.

The code:

```
async function initDb() {  
  const client = await pool.connect();  
  
  try {  
    await ensureSchema(client);
```

What's happening:

`pool.connect()` gives this operation a PostgreSQL client. `ensureSchema()` then makes sure `nodejs_applications` exists.

Why it matters:

All the database operations in this initialization process use the same client connection.

Byte 5: Preventing Duplicate Data Copies

Builds on: Byte 4

In plain terms:

Before copying anything, the script checks how many records already exist in `nodejs_applications`.

The code:

```
const { rows: countRows } = await client.query(
  'SELECT COUNT(*)::int AS count FROM nodejs_applications'
);

const targetCount = countRows[0].count;

if (targetCount > 0) {
  console.log(
    `nodejs_applications already has ${targetCount} row(s); nothing to copy.`
  );
  return { copied: 0 };
}
```

What's happening:

If the Node.js table already contains even one record, initialization stops the copying process.

Why it matters:

This is the main protection against duplicate records when the Docker container restarts.

Byte 6: Handling a Database Without the Legacy Table

Builds on: Byte 5

In plain terms:

If the old Applications table doesn't exist, there is nothing to copy. The script therefore creates three sample records instead.

The code:

```
if (
  !(await tableExists(client, 'public."Applications")) &&
```

```

    !(await tableExists(client, 'public.Applications'))
  ) {
    console.log(
      'Legacy Applications table not found; seeding nodejs_applications with sample records.'
    );

    await seedSamples(client);

    return { copied: 3, seeded: true };
  }

```

What's happening:

The script checks both possible table-name forms. If neither exists, seedSamples() provides initial data.

Why it matters:

The Node.js project can still be tested on a fresh PostgreSQL database without requiring the original .NET table.

Byte 7: Reading Data From the Original Table

Builds on: Byte 6

In plain terms:

When the legacy Applications table does exist, the script reads its application records. This is the **only point where the old table is used**, and it is read-only.

The code:

```

const legacy = await client.query(
  'SELECT "Id", "Company", "Role", "Status", "AppliedOn", "JobUrl", "Notes", "CreatedAt"
  FROM "Applications" ORDER BY "Id"'
);

```

What's happening:

The query retrieves the existing application's values ordered by ID. No INSERT, UPDATE, or DELETE is performed on "Applications".

Why it matters:

The Node.js implementation gets the same starting data while keeping the original .NET table untouched.

Byte 8: Copying Records Into the Node.js Table

Builds on: Byte 7

In plain terms:

Each legacy record is inserted into `nodejs_applications`, preserving its original ID and values.

The code:

```
for (const row of legacy.rows) {  
  await client.query(  
    `INSERT INTO nodejs_applications  
      (id, company, role, status, applied_on, job_url, notes, created_at)  
      VALUES ($1, $2, $3, $4, $5, $6, $7, $8)  
      ON CONFLICT (id) DO NOTHING`,  
    [  
      row.Id,  
      row.Company,  
      row.Role,  
      row.Status,  
      row.AppliedOn,  
      row.JobUrl,  
      row.Notes,  
      row.CreatedAt  
    ]  
  );  
}
```

What's happening:

The \$1, \$2, etc. are PostgreSQL parameter placeholders. The values from each old record are safely supplied separately. `ON CONFLICT (id) DO NOTHING` prevents an ID conflict from causing another insertion.

Why it matters:

The Node.js table starts with equivalent data without modifying the original table.

Byte 9: Using a Transaction for the Copy

Builds on: Byte 8

In plain terms:

The actual copy operation is wrapped in a database transaction. A transaction groups multiple database operations so they can either all succeed or be rolled back if something fails.

The code:

```
await client.query('BEGIN');

try {
  // copy records...

  await client.query('COMMIT');
} catch (err) {
  await client.query('ROLLBACK');
  throw err;
}
```

What's happening:

BEGIN starts the transaction. COMMIT permanently applies it. If an error occurs, ROLLBACK undoes the transaction's changes.

Why it matters:

The database won't be left with a partially completed copy if something goes wrong during initialization.

Byte 10: Resetting the ID Sequence

Builds on: Byte 8

In plain terms:

Because the copied records keep their original IDs, PostgreSQL's automatic ID counter must be synchronized with the highest existing ID.

The code:

```
await client.query(
  "SELECT setval(pg_get_serial_sequence('nodejs_applications', 'id'), " +
  "COALESCE((SELECT MAX(id) FROM nodejs_applications), 1))"
);
```

What's happening:

MAX(id) finds the largest copied ID. setval() moves the PostgreSQL sequence to that value so the next automatically generated ID doesn't collide with an existing record.

Why it matters:

Without this, a future INSERT could potentially try to reuse an ID that was already copied.

Byte 11: Seeding Sample Records

Builds on: Byte 6

In plain terms:

seedSamples() provides three sample applications when there is no legacy data available.

The code:

```
async function seedSamples(client) {  
  await client.query(  
    `INSERT INTO nodejs_applications  
      (company, role, status, applied_on, job_url, notes)  
    VALUES  
      ('Acme Labs', 'Python Intern', 'Applied', ...),  
      ('Northstar', 'Graduate Engineer', 'Interview', ...),  
      ('Contoso', 'Backend Intern', 'Rejected', ...)`  
  );  
}
```

What's happening:

Three predefined records are inserted directly into nodejs_applications. IDs are not specified, so PostgreSQL generates them.

Why it matters:

A fresh environment immediately has data that can be displayed and tested through the Node.js API.

Byte 12: Cleaning Up the Database Connection

Builds on: Bytes 4–11

In plain terms:

Whether initialization succeeds or fails, the script releases the database connection and closes the pool.

The code:

```
finally {  
  client.release();
```

```
    await pool.end();  
  }  
}
```

What's happening:

`release()` returns the connection to the pool, while `pool.end()` shuts down the pool because this initialization script has finished.

Why it matters:

Without cleanup, database connections could remain open unnecessarily.

Byte 13: Running the Script Directly

Builds on: Byte 1

In plain terms:

This block makes the file executable as a standalone Node.js script. It also reports initialization errors and exits with a failure status.

The code:

```
if (require.main === module) {  
  initDb().catch((err) => {  
    console.error(  
      'Database initialization failed:',  
      err.message  
    );  
    process.exit(1);  
  });  
}
```

What's happening:

`require.main === module` means this file itself was executed, rather than imported by another file. It then runs `initDb()` and handles any rejected promise.

Why it matters:

This allows Docker or a developer to run the initialization script directly while still allowing another module to import `initDb()`.

Byte 14: Exporting the Initialization Function

Builds on: Byte 13

In plain terms:

The final line makes `initDb()` available to other JavaScript files.

The code:

```
module.exports = { initDb };
```

What's happening:

Other code can import the function with `require()` and invoke the same initialization logic.

Why it matters:

The initialization logic isn't locked inside this file; it can be reused or tested elsewhere.

PUTTING IT TOGETHER

This script prepares the **Node.js database environment** when the project starts. It first ensures `nodejs_applications` exists, then checks whether that table already contains data. If it is empty, the script either copies the original Applications records or creates three sample records when the legacy table isn't available. The copied data is protected by a transaction and the ID sequence is synchronized afterward. Finally, the database connection is cleaned up, making the whole process safe to run repeatedly during Docker/container startup.

applicationsController.js

Byte 1: Controller Responsibility

Builds on: None — starting point

In plain terms:

This file contains the **CRUD controllers** for the Node.js API. A controller receives an HTTP request, performs the required database/validation work, and sends the HTTP response.

The code:

```
// CRUD handlers operating ONLY on the nodejs_applications table.
```

```
const pool = require('../db/pool');  
  
const { validateApplication } = require('../validation');  
  
const { mapRow } = require('../mappers');
```

What's happening:

The controller uses three dependencies: `pool` for PostgreSQL, `validateApplication` for checking incoming data, and `mapRow` for converting database rows into API responses.

Why it matters:

This keeps the API's request/response logic separate from database connection, validation, and data formatting.

Byte 2: Standard Validation Error Response

Builds on: Byte 1

In plain terms:

When the client sends invalid application data, the API returns a consistent **HTTP 400 Bad Request** response.

The code:

```
function validationProblem(res, errors) {  
  return res.status(400).json({  
    type: 'https://tools.ietf.org/html/rfc9110#section-15.5.1',  
    title: 'One or more validation errors occurred.',  
    status: 400,  
    errors  
  });  
}
```

What's happening:

The function receives Express's response object and an errors object, then sends them as JSON with status 400.

Why it matters:

All validation failures have the same response structure, making it easier for the Vue frontend to understand and display errors.

Byte 3: Handling a Missing Application

Builds on: Byte 1

In plain terms:

Several endpoints need to respond when an application ID doesn't exist. Instead of repeating that response everywhere, the controller uses one helper.

The code:

```
function notFound(res, id) {  
  return res  
    .status(404)
```

```
.json({ message: `Application ${id} not found.` });
}
```

What's happening:

The helper sends HTTP 404 Not Found with a simple JSON message.

Why it matters:

getOne, update, and remove can all reuse the same behavior instead of duplicating the response code.

Byte 4: Validating the Route ID

Builds on: Byte 3

In plain terms:

The API expects application IDs to be integers. `parseId()` checks the incoming route parameter before it is used in a database query.

The code:

```
function parseId(raw) {
  if (!/^-\d+$/.test(String(raw))) return null;
  return Number.parseInt(String(raw), 10);
}
```

What's happening:

The regular expression accepts integer values such as 5 or -5, but rejects values such as abc or 1.5. Invalid input becomes null.

Why it matters:

The API can treat malformed IDs as "not found" instead of sending an invalid value to PostgreSQL.

Byte 5: Getting All Applications

Builds on: Byte 1

In plain terms:

`getAll()` handles GET `/api/applications`. It retrieves every Node.js application and returns them in descending ID order.

The code:

```
async function getAll(req, res, next) {
  try {
    const result = await pool.query(
```

```

    'SELECT * FROM nodejs_applications ORDER BY id DESC'
  );

  res.json(result.rows.map(mapRow));
} catch (err) {
  next(err);
}
}

```

What's happening:

The database returns rows from `nodejs_applications`. Each row passes through `mapRow()` before being sent as JSON.

Why it matters:

This is the API endpoint the Vue dashboard uses to load the application's list.

Byte 6: Getting One Application

Builds on: Bytes 3–5

In plain terms:

`getOne()` handles GET `/api/applications/:id`. It validates the ID, searches for that record, and returns either the application or a 404.

The code:

```

const id = parseId(req.params.id);

if (id === null)
  return notFound(res, req.params.id);

const result = await pool.query(
  'SELECT * FROM nodejs_applications WHERE id = $1',
  [id]
);

if (result.rowCount === 0)
  return notFound(res, id);

```

```
res.json(mapRow(result.rows[0]));
```

What's happening:

req.params.id comes from the URL. The \$1 is a PostgreSQL parameter placeholder, and [id] supplies its value.

Why it matters:

The endpoint handles both malformed IDs and valid IDs that don't exist.

Byte 7: Creating an Application

Builds on: Bytes 2 and 5

In plain terms:

create() handles POST /api/applications. It validates the submitted application first and only inserts it when the data is valid.

The code:

```
const { errors, value } = validateApplication(req.body);
```

```
if (errors)
```

```
  return validationProblem(res, errors);
```

```
const result = await pool.query(
```

```
  `INSERT INTO nodejs_applications
```

```
    (company, role, status, applied_on, job_url, notes)
```

```
    VALUES ($1, $2, $3, $4, $5, $6)
```

```
    RETURNING *`,
```

```
  [
```

```
    value.company,
```

```
    value.role,
```

```
    value.status,
```

```
    value.appliedOn,
```

```
    value.jobUrl,
```

```
    value.notes
```

```
  ]
```

);

What's happening:

req.body contains the JSON sent by the frontend. Validation produces a cleaned value. That value is inserted into PostgreSQL, and RETURNING * gives back the newly created record.

Why it matters:

Invalid data never reaches the database, and the API gets the complete newly created application immediately.

Byte 8: Returning the Created Resource

Builds on: Byte 7

In plain terms:

After creating an application, the API returns the new record with HTTP 201 Created and tells the client where that resource can be found.

The code:

```
const created = mapRow(result.rows[0]);

res
  .status(201)
  .location(`/api/applications/${created.id}`)
  .json(created);
```

What's happening:

201 means a new resource was successfully created. Location contains its API URL, while the JSON body contains the created application.

Why it matters:

The client receives both the created data and its resource location in a standard REST-style response.

Byte 9: Updating an Application

Builds on: Bytes 4, 6, and 7

In plain terms:

update() handles PUT /api/applications/:id. It validates the ID and body, confirms the record exists, then updates that record.

The code:

```
const id = parseId(req.params.id);
if (id === null) return notFound(res, req.params.id);
```

```
const { errors, value } = validateApplication(req.body);
if (errors) return validationProblem(res, errors);
```

```
const existing = await pool.query(
  'SELECT * FROM nodejs_applications WHERE id = $1',
  [id]
);
```

```
if (existing.rowCount === 0)
  return notFound(res, id);
```

What's happening:

There are two separate checks: the URL ID must be valid, and the application with that ID must actually exist. Only then does the UPDATE query run.

Why it matters:

The API avoids accidentally updating a nonexistent record and keeps the same validation rules as creation.

Byte 10: Performing the Database Update

Builds on: Byte 9

In plain terms:

Once the record is confirmed to exist, the controller replaces its editable fields with the validated values.

The code:

```
const result = await pool.query(
  `UPDATE nodejs_applications
  SET company = $1, role = $2, status = $3,
    applied_on = $4, job_url = $5, notes = $6
  WHERE id = $7
  RETURNING *`,
  [
    value.company,
```

```

    value.role,
    value.status,
    value.appliedOn,
    value.jobUrl,
    value.notes,
    id
  ]
);

res.json(mapRow(result.rows[0]));

```

What's happening:

The WHERE id = \$7 ensures only the requested application is changed. RETURNING * gives the updated record back to the API.

Why it matters:

The frontend immediately receives the final state of the updated application.

Byte 11: Deleting an Application

Builds on: Bytes 4 and 6

In plain terms:

remove() handles DELETE /api/applications/:id. It validates the ID and deletes only the matching record.

The code:

```

const id = parseId(req.params.id);

if (id === null)
  return notFound(res, req.params.id);

const result = await pool.query(
  'DELETE FROM nodejs_applications WHERE id = $1',
  [id]
);

```

```
if (result.rowCount === 0)
```

```
  return notFound(res, id);
```

What's happening:

The database reports how many rows were affected. If zero rows were deleted, the requested application didn't exist.

Why it matters:

The API can distinguish a successful deletion from an attempt to delete a nonexistent application.

Byte 12: The DELETE Response

Builds on: Byte 11

In plain terms:

When deletion succeeds, the API doesn't need to return the deleted object. It simply sends HTTP 204 No Content.

The code:

```
res.status(204).end();
```

What's happening:

204 means the request succeeded but there is no response body.

Why it matters:

It gives the frontend a clear success signal without unnecessarily sending data that no longer exists.

Byte 13: Passing Unexpected Errors to Express

Builds on: Bytes 5–12

In plain terms:

Each controller uses try/catch. Expected problems such as validation or missing records are handled directly; unexpected database/server errors are passed to Express's error-handling middleware.

The code:

```
try {  
  // controller work  
} catch (err) {  
  next(err);  
}
```

What's happening:

`next(err)` tells Express that an error occurred and allows the application's centralized error handler to deal with it.

Why it matters:

Database errors don't need to be manually handled inside every controller function.

Byte 14: Exporting the CRUD Handlers

Builds on: Bytes 5–13

In plain terms:

The final line makes these controller functions available to the routing file.

The code:

```
module.exports = {  
  getAll,  
  getOne,  
  create,  
  update,  
  remove  
};
```

What's happening:

The routes file can import these functions and connect them to HTTP methods and URLs.

Why it matters:

This creates a clean separation: **routes decide which controller runs, while controllers decide what the request actually does.**

PUTTING IT TOGETHER

This file is the **controller layer** of your Node.js backend. A request first reaches a route, which calls one of these handlers such as `getAll`, `create`, `update`, or `remove`. The controller validates input when necessary, communicates with PostgreSQL through `pool`, converts database rows using `mapRow`, and sends the appropriate HTTP response. Invalid requests get 400, missing applications get 404, successful creation gets 201, successful deletion gets 204, and unexpected errors are passed to Express's error handler. Most importantly for your project, every CRUD query here operates on **`nodejs_applications`**, not the original `Applications` table.

pool.js

Byte 1: Purpose of the Database Pool

Builds on: None — starting point

In plain terms:

This file creates the **PostgreSQL connection pool** used by the Node.js backend. A connection pool keeps reusable database connections available so every API request doesn't have to create a brand-new database connection.

The code:

```
const { Pool, types } = require('pg');

const pool = new Pool(buildConfig());

module.exports = pool;
```

What's happening:

pg is the Node.js PostgreSQL library. Pool manages database connections, and the created pool is exported so controllers and other database code can use it.

Why it matters:

Without this pool, the backend would not have a convenient way to communicate with PostgreSQL.

Byte 2: Reading PostgreSQL Connection Settings

Builds on: Byte 1

In plain terms:

The connection details are taken from **environment variables** instead of being permanently hardcoded. This lets the same code work in local development and inside Docker.

The code:

```
process.env.PGHOST
process.env.PGPORT
process.env.PGDATABASE
process.env.PGUSER
process.env.PGPASSWORD
```

What's happening:

process.env gives Node.js access to environment variables. For example, PGPORT specifies the PostgreSQL port and PGDATABASE specifies the database name.

Why it matters:

You can change database settings without changing the source code.

Byte 3: Supporting DATABASE_URL

Builds on: Byte 2

In plain terms:

The application supports two ways of specifying the PostgreSQL connection: a single DATABASE_URL, or individual PostgreSQL environment variables.

The code:

```
function buildConfig() {  
  if (process.env.DATABASE_URL) {  
    return { connectionString: process.env.DATABASE_URL };  
  }  
  
  return {  
    host: process.env.PGHOST || '127.0.0.1',  
    port: Number(process.env.PGPORT || 5432),  
    database: process.env.PGDATABASE || 'placement_tracker',  
    user: process.env.PGUSER || 'postgres',  
    password: process.env.PGPASSWORD || 'postgres'  
  };  
}
```

What's happening:

If DATABASE_URL exists, pg uses it directly. Otherwise, the function builds the configuration from individual settings.

Why it matters:

This makes the backend flexible across different environments without changing the database code.

Byte 4: Default PostgreSQL Configuration

Builds on: Byte 3

In plain terms:

If an environment variable isn't provided, the code uses sensible defaults for your project.

The code:

```
host: process.env.PGHOST || '127.0.0.1',
port: Number(process.env.PGPORT || 5432),
database: process.env.PGDATABASE || 'placement_tracker',
user: process.env.PGUSER || 'postgres',
password: process.env.PGPASSWORD || 'postgres'
```

What's happening:

The `||` means "use the environment value if it exists; otherwise use this default." So PostgreSQL defaults to port 5432 and database `placement_tracker`.

Why it matters:

The application can run with minimal configuration during development while still allowing Docker or deployment environments to provide their own values.

Byte 5: Creating the PostgreSQL Pool

Builds on: Bytes 1–4

In plain terms:

Once the database configuration is ready, the application creates the actual connection pool.

The code:

```
const pool = new Pool(buildConfig());
```

What's happening:

`buildConfig()` produces the database settings, and `new Pool(...)` creates a pool that `pool.query()` and `pool.connect()` can use.

Why it matters:

This pool becomes the shared database access point for the Node.js backend.

Byte 6: PostgreSQL Timestamp Handling

Builds on: Byte 1

In plain terms:

The project specifically controls how PostgreSQL's timestamp without time zone values are read. Instead of allowing pg to automatically convert them into JavaScript Date objects, the value is kept as a string.

The code:

```
// 1114 = timestamp without time zone
```

```
types.setTypeParser(1114, (value) => value);
```

What's happening:

PostgreSQL assigns type ID 1114 to timestamp without time zone. This custom parser tells pg to return the original timestamp text unchanged.

Why it matters:

The project wants to preserve calendar values such as:

2026-09-16 00:00:00

without an automatic timezone conversion potentially shifting the date.

Byte 7: Exporting the Pool

Builds on: Byte 5

In plain terms:

The completed pool is exported so the rest of the backend can use the same database connection manager.

The code:

```
module.exports = pool;
```

What's happening:

Another file can import it with:

```
const pool = require('../db/pool');
```

and then execute queries such as:

```
await pool.query('SELECT * FROM nodejs_applications');
```

Why it matters:

Controllers and database modules don't need to create their own PostgreSQL connection configuration. They reuse this shared pool.

PUTTING IT TOGETHER

This file is the **database connection layer** of your Node.js backend. It uses the pg library to create a reusable PostgreSQL connection pool and gets its connection details from environment variables or DATABASE_URL. Default values allow it to connect to your placement_tracker database during normal development. It also preserves PostgreSQL timestamp without time zone values as strings to avoid unwanted date shifting. Finally, the pool is exported so your controllers and database initialization code can execute PostgreSQL queries through the same connection manager.

schema.js

Byte 1: Purpose of the Schema Module

Builds on: None — starting point

In plain terms:

This file defines the **database structure** for the Node.js implementation. Its job is to make sure the `nodejs_applications` table exists with the correct columns, data types, and rules.

The code:

```
const pool = require('./pool');

const CREATE_TABLE_SQL = `
CREATE TABLE IF NOT EXISTS nodejs_applications (
  ...
);`;
```

What's happening:

The file imports the PostgreSQL pool and stores the SQL command used to create the table.

Why it matters:

Without this schema, the Node.js API would have no table in which to store application records.

Byte 2: Preventing Unnecessary Table Recreation

Builds on: Byte 1

In plain terms:

The table is created only if it doesn't already exist. This makes the operation **idempotent** — running it multiple times doesn't destroy or recreate existing data.

The code:

```
CREATE TABLE IF NOT EXISTS nodejs_applications (
```

What's happening:

PostgreSQL checks whether `nodejs_applications` already exists. If it does, the `CREATE TABLE` operation does nothing.

Why it matters:

Your Docker startup process can safely run `ensureSchema()` every time without deleting the existing Node.js application data.

Byte 3: Defining the Primary Key

Builds on: Byte 2

In plain terms:

Every application needs a unique identifier. The id column provides that identifier and PostgreSQL automatically generates it for new records.

The code:

id SERIAL PRIMARY KEY,

What's happening:

PRIMARY KEY means every ID must be unique. SERIAL creates an automatically incrementing value for new rows.

Why it matters:

The API uses this ID to retrieve, update, and delete individual applications.

Byte 4: Defining Company and Role

Builds on: Byte 3

In plain terms:

Each application must contain a company and job role. Both fields are limited to 100 characters and cannot be empty at the database level.

The code:

company VARCHAR(100) NOT NULL,

role VARCHAR(100) NOT NULL,

What's happening:

VARCHAR(100) limits the stored text to 100 characters. NOT NULL means PostgreSQL requires a value.

Why it matters:

The database itself enforces these basic requirements, providing a second layer of protection beyond API validation.

Byte 5: Restricting Application Status

Builds on: Byte 4

In plain terms:

The application status cannot be arbitrary text. PostgreSQL only accepts the five statuses defined by the project.

The code:

status VARCHAR(20) NOT NULL

```
CHECK (  
    status IN (  
        'Wishlist',  
        'Applied',  
        'Interview',  
        'Offer',  
        'Rejected'  
    )  
),
```

What's happening:

The CHECK constraint rejects any status outside this list.

Why it matters:

It prevents invalid values such as "Pending" or "Random" from entering the database.

Byte 6: Optional Application Fields

Builds on: Byte 5

In plain terms:

Some application information isn't mandatory. The applied date, job URL, and notes can therefore be left empty.

The code:

```
applied_on TIMESTAMP WITHOUT TIME ZONE NULL,  
job_url TEXT NULL,  
notes VARCHAR(1000) NULL,
```

What's happening:

NULL means the field can have no value. The notes field is limited to 1000 characters.

Why it matters:

For example, a Wishlist application may not have an application date or URL yet.

Byte 7: Automatically Recording Creation Time

Builds on: Byte 6

In plain terms:

Every application gets a creation timestamp automatically when it is inserted.

The code:

created_at TIMESTAMP WITHOUT TIME ZONE

NOT NULL DEFAULT NOW()

What's happening:

DEFAULT NOW() tells PostgreSQL to automatically use the current database time when the application doesn't provide a created_at value.

Why it matters:

The backend doesn't need to manually calculate the creation time for every new application.

Byte 8: Executing the Schema

Builds on: Bytes 1–7

In plain terms:

ensureSchema() is the function that actually sends the table-creation SQL to PostgreSQL.

The code:

```
async function ensureSchema(client) {  
  const runner = client || pool;  
  await runner.query(CREATE_TABLE_SQL);  
}
```

What's happening:

If a database client is provided, the function uses that client. Otherwise, it uses the shared connection pool imported at the top.

Why it matters:

This makes the function reusable: it can participate in an existing database operation or simply use the normal pool.

Byte 9: Exporting the Schema Function

Builds on: Byte 8

In plain terms:

The module exposes ensureSchema() and the SQL definition so other parts of the application can use them.

The code:

```
module.exports = {  
  ensureSchema,  
  CREATE_TABLE_SQL  
};
```

What's happening:

Other files can import ensureSchema and call it during database initialization.

Why it matters:

Your init-db.js script can ensure the Node.js table exists without duplicating the table definition.

PUTTING IT TOGETHER

This file is responsible for defining the **nodejs_applications database table**. The SQL establishes the columns, primary key, allowed statuses, optional fields, and automatic creation timestamp. IF NOT EXISTS makes the operation safe to repeat, which is important during Docker startup. ensureSchema() executes that definition using either an existing database client or the shared PostgreSQL pool. The important isolation point is that this module creates and works with **nodejs_applications only**; it does not modify the original Applications table.

routes/applications.js

Byte 1: Purpose of the Routes File

Builds on: None — starting point

In plain terms:

This file defines the **API routes** for applications. A route connects an HTTP method and URL to the controller function that should handle the request.

The code:

```
const express = require('express');  
  
const controller = require('../controllers/applicationsController');
```

```
const router = express.Router();
```

What's happening:

Express provides the routing system, while the controller contains the actual CRUD logic. express.Router() creates a smaller router that can hold these application-related routes.

Why it matters:

It keeps URL definitions separate from the database and business logic.

Byte 2: GET All Applications Route

Builds on: Byte 1

In plain terms:

This route handles requests to retrieve all applications.

The code:

```
router.get('/', controller.getAll);
```

What's happening:

When a GET request reaches this router at /, Express calls controller.getAll.

In your complete API, this becomes:

GET /api/applications

Why it matters:

It connects the frontend's request for the application list to the controller that actually queries PostgreSQL.

Byte 3: GET One Application Route

Builds on: Byte 2

In plain terms:

This route retrieves one application using its ID from the URL.

The code:

```
router.get('/:id', controller.getOne);
```

What's happening:

:id is a **route parameter**. For example:

GET /api/applications/5

makes req.params.id equal to 5, and Express calls controller.getOne.

Why it matters:

The same route definition can handle any application ID instead of creating a separate route for every record.

Byte 4: POST Route for Creating Applications

Builds on: Byte 1

In plain terms:

This route handles requests that create a new application.

The code:

```
router.post('/', controller.create);
```

What's happening:

A POST request to:

/api/applications

is passed to `controller.create`, which validates the data and inserts it into PostgreSQL.

Why it matters:

The route decides **where the request goes**; the controller decides **how the application is created**.

Byte 5: PUT Route for Updating Applications

Builds on: Byte 3

In plain terms:

This route updates an existing application identified by its ID.

The code:

```
router.put('/:id', controller.update);
```

What's happening:

For example:

PUT /api/applications/5

causes Express to call `controller.update` with `req.params.id = 5`.

Why it matters:

It connects the frontend's edit operation to the controller responsible for updating the corresponding database record.

Byte 6: DELETE Route

Builds on: Byte 3

In plain terms:

This route deletes an application identified by its ID.

The code:

```
router.delete('/:id', controller.remove);
```

What's happening:

For example:

DELETE /api/applications/5

calls `controller.remove`, which deletes the matching record from `nodejs_applications`.

Why it matters:

It completes the CRUD operations by providing the delete operation.

Byte 7: Exporting the Router

Builds on: Bytes 1–6

In plain terms:

After defining all the routes, the router is exported so the main Express application can use it.

The code:

```
module.exports = router;
```

What's happening:

Another file can import this router and mount it under a common path such as:

```
/api/applications
```

The `/` and `/:id` paths defined here are then combined with that prefix.

Why it matters:

Without exporting the router, the main Express application wouldn't be able to connect these routes to the API.

PUTTING IT TOGETHER

This file is the **routing layer** for your application API. It maps the five CRUD operations to controller functions: GET all, GET one, POST, PUT, and DELETE. The routes themselves do not contain PostgreSQL queries or validation; they simply decide **which controller should handle each request**. The exported router is then mounted by the main Express application under `/api/applications`.

app.js

Byte 1: Purpose of the Routes File

Builds on: None — starting point

In plain terms:

This file defines the **API routes** for applications. A route connects an HTTP method and URL to the controller function that should handle the request.

The code:

```
const express = require('express');  
  
const controller = require('../controllers/applicationsController');  
  
  
const router = express.Router();
```

What's happening:

Express provides the routing system, while the controller contains the actual CRUD logic. `express.Router()` creates a smaller router that can hold these application-related routes.

Why it matters:

It keeps URL definitions separate from the database and business logic.

Byte 2: GET All Applications Route

Builds on: Byte 1

In plain terms:

This route handles requests to retrieve all applications.

The code:

```
router.get('/', controller.getAll);
```

What's happening:

When a GET request reaches this router at `/`, Express calls `controller.getAll`.

In your complete API, this becomes:

GET `/api/applications`

Why it matters:

It connects the frontend's request for the application list to the controller that actually queries PostgreSQL.

Byte 3: GET One Application Route

Builds on: Byte 2

In plain terms:

This route retrieves one application using its ID from the URL.

The code:

```
router.get('/:id', controller.getOne);
```

What's happening:

`:id` is a **route parameter**. For example:

GET `/api/applications/5`

makes `req.params.id` equal to `5`, and Express calls `controller.getOne`.

Why it matters:

The same route definition can handle any application ID instead of creating a separate route for every record.

Byte 4: POST Route for Creating Applications

Builds on: Byte 1

In plain terms:

This route handles requests that create a new application.

The code:

```
router.post('/', controller.create);
```

What's happening:

A POST request to:

/api/applications

is passed to controller.create, which validates the data and inserts it into PostgreSQL.

Why it matters:

The route decides **where the request goes**; the controller decides **how the application is created**.

Byte 5: PUT Route for Updating Applications

Builds on: Byte 3

In plain terms:

This route updates an existing application identified by its ID.

The code:

```
router.put('/:id', controller.update);
```

What's happening:

For example:

PUT /api/applications/5

causes Express to call controller.update with req.params.id = 5.

Why it matters:

It connects the frontend's edit operation to the controller responsible for updating the corresponding database record.

Byte 6: DELETE Route

Builds on: Byte 3

In plain terms:

This route deletes an application identified by its ID.

The code:

```
router.delete('/:id', controller.remove);
```

What's happening:

For example:

```
DELETE /api/applications/5
```

calls `controller.remove`, which deletes the matching record from `nodejs_applications`.

Why it matters:

It completes the CRUD operations by providing the delete operation.

Byte 7: Exporting the Router

Builds on: Bytes 1–6

In plain terms:

After defining all the routes, the router is exported so the main Express application can use it.

The code:

```
module.exports = router;
```

What's happening:

Another file can import this router and mount it under a common path such as:

```
/api/applications
```

The `/` and `/:id` paths defined here are then combined with that prefix.

Why it matters:

Without exporting the router, the main Express application wouldn't be able to connect these routes to the API.

PUTTING IT TOGETHER

This file is the **routing layer** for your application API. It maps the five CRUD operations to controller functions: GET all, GET one, POST, PUT, and DELETE. The routes themselves do not contain PostgreSQL queries or validation; they simply decide **which controller should handle each request**. The exported router is then mounted by the main Express application under `/api/applications`.

mappers.js

Byte 1: Purpose of the Mapper

Builds on: None — starting point

In plain terms:

This file is a **data mapper**. Its job is to convert PostgreSQL database rows, which use snake_case, into the camelCase JSON format expected by your Vue frontend.

The code:

```
// PostgreSQL:
```

```
// applied_on, job_url, created_at
```

```
// API JSON:
```

```
// appliedOn, jobUrl, createdAt
```

What's happening:

The database and frontend use different naming conventions. mapRow() acts as the bridge between them.

Why it matters:

The frontend can keep using its existing API contract without changing the PostgreSQL column names or the Node.js database structure.

Byte 2: Handling Missing Date Values

Builds on: Byte 1

In plain terms:

toIsoDateTime() first handles cases where a timestamp doesn't exist. In those cases, it returns null.

The code:

```
function toIsoDateTime(value) {  
  if (value === null || value === undefined) return null;
```

What's happening:

The function checks for both null and undefined before doing any date conversion.

Why it matters:

Optional fields such as applied_on can be empty. This prevents the mapper from trying to process a missing value.

Byte 3: Converting JavaScript Date Objects

Builds on: Byte 2

In plain terms:

If PostgreSQL gives the value to JavaScript as a Date object, the mapper converts it into an ISO timestamp string.

The code:

```
if (value instanceof Date) {  
    return value.toISOString();  
}
```

What's happening:

instanceof Date checks whether the value is a JavaScript Date. toISOString() converts it into a standard format such as:

2026-09-16T00:00:00.000Z

Why it matters:

The API consistently returns dates as strings that can easily be consumed by the frontend.

Byte 4: Handling PostgreSQL Timestamp Strings

Builds on: Byte 3

In plain terms:

In this project, PostgreSQL timestamps are intentionally returned as strings. The mapper converts the PostgreSQL format into an ISO-style format.

The code:

```
const text = String(value);  
  
if (/^\d{4}-\d{2}-\d{2} \d{2}:\d{2}:\d{2}/.test(text)) {  
    return text.replace(' ', 'T');  
}
```

What's happening:

A PostgreSQL timestamp might look like:

2026-09-16 00:00:00

The code recognizes that format and changes the space between the date and time to T:

2026-09-16T00:00:00

Why it matters:

This matches the date-time representation expected by the existing API/frontend.

Byte 5: Mapping a Database Row

Builds on: Bytes 1–4

In plain terms:

mapRow() converts one complete PostgreSQL row into the JSON object returned by the API.

The code:

```
function mapRow(row) {  
  if (!row) return null;  
  
  return {  
    id: row.id,  
    company: row.company,  
    role: row.role,  
    status: row.status,  
    appliedOn: row.applied_on  
      ? toIsoDateTime(row.applied_on)  
      : null,  
    jobUrl: row.job_url,  
    notes: row.notes,  
    createdAt: toIsoDateTime(row.created_at)  
  };  
}
```

What's happening:

Most fields are copied directly. The fields requiring transformation are renamed:

applied_on → appliedOn

job_url → jobUrl

created_at → createdAt

The date fields are also passed through toIsoDateTime().

Why it matters:

This keeps the database representation separate from the API representation.

Byte 6: Handling a Missing Row

Builds on: Byte 5

In plain terms:

If `mapRow()` receives no database row, it returns null instead of trying to access properties from an empty value.

The code:

```
if (!row) return null;
```

What's happening:

The function exits immediately when row is missing.

Why it matters:

It prevents errors such as trying to access `row.id` when row doesn't exist.

Byte 7: Exporting the Mapper

Builds on: Byte 5

In plain terms:

The `mapRow()` function is exported so controllers can use it when returning database results to the frontend.

The code:

```
module.exports = { mapRow };
```

What's happening:

A controller can import it:

```
const { mapRow } = require('../mappers');
```

and then use:

```
res.json(mapRow(result.rows[0]));
```

Why it matters:

The controller doesn't need to manually convert database column names every time it sends a response.

PUTTING IT TOGETHER

This file sits between **PostgreSQL and the API response**. PostgreSQL uses names such as `applied_on` and `created_at`, while the existing Vue frontend expects `appliedOn` and `createdAt`. `mapRow()` performs that conversion, while `toISOString()` handles the timestamp formatting. This means the database can keep its `snake_case` structure while the frontend continues using the existing `camelCase` API contract.

server.js

Byte 1: Purpose of the Server Entry Point

Builds on: None — starting point

In plain terms:

This file is the **entry point** of your Node.js backend. Its job is to prepare the database, create the Express application, and start the API server.

The code:

```
const { createApp } = require('./app');  
  
const pool = require('./db/pool');  
  
const { ensureSchema } = require('./db/schema');
```

What's happening:

The file imports the three pieces it needs: the Express app factory, the PostgreSQL pool, and the function that ensures the database table exists.

Why it matters:

This file connects the major backend pieces together when the Node.js server starts.

Byte 2: Configuring the API Port and Host

Builds on: Byte 1

In plain terms:

The server needs to know **where to listen** for incoming requests. Your Node.js API uses port 3000 internally and listens only on the local machine.

The code:

```
const PORT = Number(process.env.PORT || 3000);  
  
const HOST = process.env.HOST || '127.0.0.1';
```

What's happening:

Environment variables can override the defaults. Otherwise, the API uses:

Host: 127.0.0.1

Port: 3000

In your Docker architecture, NGINX receives public traffic on port 8080 and forwards API requests to this Node.js server.

Why it matters:

Keeping Node on 3000 separates the application server from NGINX's public-facing 8080 port.

Byte 3: The Main Startup Function

Builds on: Bytes 1–2

In plain terms:

main() defines the startup sequence. The database is prepared first, then the Express application is created.

The code:

```
async function main() {  
  await ensureSchema();  
  
  const app = createApp();
```

What's happening:

await ensureSchema() makes sure nodejs_ applications exists before the API starts. Then createApp() creates the configured Express application.

Why it matters:

The API doesn't start before its required database table has been prepared.

Byte 4: Starting the HTTP Server

Builds on: Byte 3

In plain terms:

After the database and Express application are ready, app.listen() starts accepting HTTP requests.

The code:

```
app.listen(PORT, HOST, () => {  
  console.log(  
    `Placement Tracker API listening on http://${HOST}:${PORT}`  
  );  
});
```

What's happening:

Express starts listening on 127.0.0.1:3000. The callback runs once the server successfully starts.

Why it matters:

This is the point where your Node.js API becomes available to receive requests from NGINX.

Byte 5: Starting the Application

Builds on: Byte 4

In plain terms:

The file calls `main()` to actually begin the startup process.

The code:

```
main().catch((err) => {  
  console.error('Failed to start API:', err.message);  
  pool.end().finally(() => process.exit(1));  
});
```

What's happening:

Because `main()` is asynchronous, `.catch()` handles any startup failure. If something goes wrong, the error is printed, the PostgreSQL pool is closed, and the process exits with status 1.

Why it matters:

A failed API startup doesn't leave database connections running or make the system appear healthy when the backend actually failed.

PUTTING IT TOGETHER

`server.js` is the **starting point of your Node.js backend**. When it runs, it first ensures the `nodejs_applications` table exists, then creates the Express application and starts it on `127.0.0.1:3000`. NGINX sits in front of this server and exposes the application through port `8080`. If startup fails at any stage, the error is handled and the PostgreSQL connection pool is cleaned up before the process exits.

validation.js

Byte 1: Purpose of the Validation Module

Builds on: None — starting point

In plain terms:

This file checks whether data sent to the API is a valid placement application before it reaches PostgreSQL. It also cleans the data into the exact format the rest of the backend expects.

The code:

```
const ALLOWED_STATUSES = [  
  'Wishlist',  
  'Applied',
```

```
'Interview',  
'Offer',  
'Rejected'  
];
```

```
function validateApplication(body) {  
  const errors = {};  
  // validation...  
}
```

What's happening:

validateApplication() receives the request body and checks fields such as company, role, status, date, URL, and notes.

Why it matters:

It prevents invalid data from being inserted or updated and keeps the Node.js implementation compatible with the existing Vue frontend.

Byte 2: Defining Allowed Statuses

Builds on: Byte 1

In plain terms:

An application can only have one of five predefined statuses.

The code:

```
const ALLOWED_STATUSES = [  
  'Wishlist',  
  'Applied',  
  'Interview',  
  'Offer',  
  'Rejected'  
];
```

What's happening:

The array acts as the single list of valid status values. Later, .includes() checks whether the submitted status is in this list.

Why it matters:

It prevents inconsistent values such as "Pending" or "Selected" from entering the application data.

Byte 3: Detecting Missing Values

Builds on: Byte 1

In plain terms:

isMissing() identifies values that should be considered absent, including undefined, null, and an empty string.

The code:

```
function isMissing(value) {  
  return value === undefined ||  
    value === null ||  
    (typeof value === 'string' && value.length === 0);  
}
```

What's happening:

The function handles the common ways a field can arrive without a value.

Why it matters:

The validation logic can reuse one consistent definition of "missing" instead of repeating these checks.

Byte 4: Normalizing Optional Fields

Builds on: Byte 3

In plain terms:

Optional fields such as jobUrl and notes are cleaned before storage. Empty strings become null, while non-empty strings are trimmed.

The code:

```
function normalizeOptional(value) {  
  if (value === undefined || value === null) return null;  
  if (typeof value !== 'string') return value;  
  return value.trim() === "" ? null : value.trim();  
}
```

What's happening:

For example, " https://example.com " becomes "https://example.com", while " " becomes null.

Why it matters:

The database doesn't end up storing meaningless whitespace as actual content.

Byte 5: Validating the Application Date

Builds on: Byte 1

In plain terms:

`parseAppliedOn()` checks that the application date starts with a valid YYYY-MM-DD date and returns that calendar date in normalized form.

The code:

```
function parseAppliedOn(value) {  
  const text = String(value).trim();  
  const match = text.match(/^(\\d{4})-(\\d{2})-(\\d{2})/);  
  
  if (!match) {  
    return { error: 'Applied date is invalid.' };  
  }  
  
  const [, y, m, d] = match;  
  // date validity check...  
  
  return { date: `${y}-${m}-${d}` };  
}
```

What's happening:

The regular expression extracts year, month, and day. The function then checks whether those numbers represent a real calendar date.

Why it matters:

The project stores `AppliedOn` as a plain calendar date, avoiding unwanted timezone shifts.

Byte 6: Checking Whether the Date Actually Exists

Builds on: Byte 5

In plain terms:

Matching the YYYY-MM-DD pattern isn't enough. A date like 2026-02-31 has the correct shape but isn't a real date, so the function performs an additional check.

The code:

```
const check = new Date(`${y}-${m}-${d}T00:00:00Z`);
```

```

if (
  Number.isNaN(check.getTime()) ||
  check.getUTCFullYear() !== Number(y) ||
  check.getUTCMonth() + 1 !== Number(m) ||
  check.getUTCDate() !== Number(d)
) {
  return { error: 'Applied date is invalid.' };
}

```

What's happening:

The constructed date is compared back against the original year, month, and day. If JavaScript normalizes an invalid date into a different date, the validation fails.

Why it matters:

This catches invalid calendar dates rather than accepting them simply because their text looks correct.

Byte 7: Validating Job URLs

Builds on: Byte 4

In plain terms:

If a job URL is supplied, it must be a valid absolute http or https URL.

The code:

```

function isValidJobUrl(value) {
  try {
    const url = new URL(value);
    return url.protocol === 'http:' ||
      url.protocol === 'https:';
  } catch {
    return false;
  }
}

```

What's happening:

JavaScript's URL parser checks whether the value is a valid URL. The protocol check then restricts it to HTTP or HTTPS.

Why it matters:

Values such as invalid text or unsupported protocols are rejected before reaching the database.

Byte 8: Creating the Validation Error Structure

Builds on: Bytes 2–7

In plain terms:

`validateApplication()` collects all validation problems into an errors object rather than stopping at the first problem.

The code:

```
const errors = {};  
  
const add = (field, message) => {  
  if (!errors[field]) errors[field] = [];  
  errors[field].push(message);  
};
```

What's happening:

Each field can have an array of messages. For example:

```
{  
  Company: ['Company is required.'],  
  Role: ['Role must be 2-100 characters.'],  
}
```

Why it matters:

The frontend can receive all relevant validation problems in one response instead of fixing them one at a time.

Byte 9: Validating the Request Body

Builds on: Byte 8

In plain terms:

Before checking individual fields, the function verifies that the request body itself is a usable object.

The code:

```
if (  
  body === undefined ||
```

```

body === null ||
typeof body !== 'object' ||
Array.isArray(body)
) {
  return {
    errors: { ": ['Invalid request body.'],",
    value: null
  };
}

```

What's happening:

Values such as null, a string, number, or array are rejected because the API expects an object containing application fields.

Why it matters:

It prevents the rest of the validation code from operating on an unexpected data structure.

Byte 10: Company Validation

Builds on: Byte 8

In plain terms:

The company name is required and must contain between 2 and 100 non-whitespace characters.

The code:

```

if (isMissing(company)) {
  add('Company', 'Company is required.');
```



```

}
```



```

if (
  typeof company !== 'string' ||
  company.trim().length < 2 ||
  company.trim().length > 100
) {
  add('Company', 'Company must be 2-100 characters.');
```



```

}
```

What's happening:

The value is checked both for presence and for its trimmed length.

Why it matters:

A value containing only spaces doesn't pass the meaningful-content requirement.

Byte 11: Role Validation

Builds on: Byte 10

In plain terms:

The role follows the same basic rule as the company: it is required and must be 2–100 characters after trimming.

The code:

```
if (isMissing(role)) {  
    add('Role', 'Role is required.');
```



```
}  
  
if (  
    typeof role !== 'string' ||  
    role.trim().length < 2 ||  
    role.trim().length > 100  
) {  
    add('Role', 'Role must be 2-100 characters.');
```



```
}
```

What's happening:

The validator checks both presence and length.

Why it matters:

This prevents empty, whitespace-only, or excessively long job-role values.

Byte 12: Status Validation

Builds on: Byte 2

In plain terms:

The status must exist and must be one of the predefined statuses.

The code:

```
if (isMissing(status)) {
```

```

    add('Status', 'Status is required.');
```

```

}
```

```

if (!ALLOWED_STATUSES.includes(status)) {
    add(
        'Status',
        'Status must be one of: Wishlist, Applied, Interview, Offer, Rejected.'
    );
}
```

What's happening:

The first check handles a missing status. The second checks whether the supplied value exists in ALLOWED_STATUSES.

Why it matters:

It keeps status values consistent throughout the application.

Byte 13: Conditional Applied-Date Validation

Builds on: Bytes 5 and 12

In plain terms:

AppliedOn has a special rule: it is required for normal applications but **not required when the status is Wishlist**.

The code:

```

const appliedOnMissing =
    appliedOn === undefined ||
    appliedOn === null ||
    String(appliedOn).trim() === "";

if (appliedOnMissing) {
    if (status !== 'Wishlist') {
        add(
            'AppliedOn',
            'Applied date is required unless status is Wishlist.'
        );
    }
}
```

```
}  
}
```

What's happening:

The validator first determines whether the date is missing. If it is missing and the status isn't Wishlist, an error is added.

Why it matters:

A job that is only saved as a wishlist item doesn't necessarily have an application date yet.

Byte 14: Parsing a Supplied Applied Date

Builds on: Byte 13

In plain terms:

If an AppliedOn value is supplied, it must pass parseAppliedOn() before being accepted.

The code:

```
const parsed = parseAppliedOn(appliedOn);
```

```
if (parsed.error) {  
  add('AppliedOn', parsed.error);  
} else {  
  normalizedAppliedOn = parsed.date;  
}
```

What's happening:

A valid date is stored in normalized YYYY-MM-DD form. An invalid date produces a validation error.

Why it matters:

The database receives a consistent date format instead of arbitrary date strings.

Byte 15: Optional Job URL Validation

Builds on: Bytes 4 and 7

In plain terms:

The job URL isn't mandatory, but when provided it must be a valid HTTP/HTTPS URL.

The code:

```
const normalizedJobUrl = normalizeOptional(jobUrl);
```

```

if (normalizedJobUrl !== null) {
  if (
    typeof normalizedJobUrl !== 'string' ||
    !isValidJobUrl(normalizedJobUrl)
  ) {
    add(
      'JobUrl',
      'Please enter a valid URL (e.g. https://example.com/job).'
    );
  }
}

```

What's happening:

The value is first normalized. Only a non-empty value is then passed through URL validation.

Why it matters:

An empty optional URL is accepted, but a supplied invalid URL is rejected.

Byte 16: Optional Notes Validation

Builds on: Byte 4

In plain terms:

Notes are optional, but if supplied they cannot exceed 1000 characters.

The code:

```

const normalizedNotes = normalizeOptional(notes);

if (normalizedNotes !== null) {
  if (typeof normalizedNotes !== 'string') {
    add('Notes', 'Notes cannot be longer than 1000 characters.');
```

```

  } else if (normalizedNotes.length > 1000) {
    add('Notes', 'Notes cannot be longer than 1000 characters.');
```

```

  }
}

```

What's happening:

Empty notes become null. Otherwise, the validator checks the type and maximum length.

Why it matters:

It prevents oversized or invalid notes from being stored.

Byte 17: Returning Validation Errors

Builds on: Bytes 8–16

In plain terms:

After all fields have been checked, the function determines whether any errors were collected.

The code:

```
if (Object.keys(errors).length > 0) {  
  return {  
    errors,  
    value: null  
  };  
}
```

What's happening:

`Object.keys(errors)` returns the fields that contain errors. If there is at least one, the function returns the complete error collection and no normalized value.

Why it matters:

The controller can detect errors and return HTTP 400 without sending invalid data to PostgreSQL.

Byte 18: Returning the Normalized Application

Builds on: Bytes 4–16

In plain terms:

If every validation rule passes, the function returns a cleaned application object ready for the database.

The code:

```
return {  
  errors: null,  
  value: {  
    company: company.trim(),  
    role: role.trim(),  
    status,
```

```
    appliedOn: normalizedAppliedOn,  
    jobUrl: normalizedJobUrl,  
    notes: normalizedNotes  
  }  
};
```

What's happening:

Company and role are trimmed, optional empty values are represented as null, and the date has already been normalized.

Why it matters:

The controller gets one reliable, normalized object instead of having to clean the data itself.

Byte 19: Exporting the Validator

Builds on: Bytes 17–18

In plain terms:

The module exports the validation function and the allowed status list so other backend files can use them.

The code:

```
module.exports = {  
  validateApplication,  
  ALLOWED_STATUSES  
};
```

What's happening:

Your controller can import `validateApplication()` before performing INSERT or UPDATE operations.

Why it matters:

Validation remains centralized instead of being duplicated across multiple API endpoints.

PUTTING IT TOGETHER

This module is the **validation and normalization layer** of your Node.js backend. The controller sends the incoming request body to `validateApplication()`, which checks the structure, required fields, status, date, URL, and notes. If anything is invalid, it returns an errors object; otherwise, it returns a cleaned value ready for PostgreSQL. The important design point is that validation happens **before the database operation**, while normalization ensures the stored values have a consistent format.

applications.test.js

Byte 1: Purpose of the Test File

Builds on: None — starting point

In plain terms:

This file contains **API integration tests** for the Node.js Placement Tracker. It checks CRUD operations, validation, error handling, database persistence, and—importantly—that the original Applications table is never modified.

The code:

```
const { describe, it, before, after } = require('node:test');  
const assert = require('node:assert/strict');  
const request = require('supertest');
```

```
const { createApp } = require('../src/app');  
const pool = require('../src/db/pool');  
const { ensureSchema } = require('../src/db/schema');
```

What's happening:

The test uses Node's built-in test framework, assert for checking results, and **Supertest** for sending HTTP requests to the Express application. It also imports the application, PostgreSQL connection pool, and database schema setup.

Why it matters:

These tests verify the system as a whole instead of testing only individual functions. They can catch problems between the API, validation, and PostgreSQL.

Byte 2: Creating the Test Application

Builds on: Byte 1

In plain terms:

The tests need an Express application to send requests to. `createApp()` creates that application without starting a real HTTP server.

The code:

```
const app = createApp();
```

What's happening:

The Express app is created directly and passed to Supertest. Supertest can communicate with this app internally.

Why it matters:

The tests don't need to start node server.js or occupy port 3000. This makes the tests faster and easier to isolate.

Byte 3: Creating a Unique Test Marker

Builds on: Byte 2

In plain terms:

Every test record gets a unique company name. This makes it easy to identify and remove records created specifically by the test suite.

The code:

```
const MARKER = `NodeTest ${Date.now()}`;
```

```
function payload(overrides = {}) {  
  return {  
    company: `${MARKER} Co`,  
    role: 'Backend Intern',  
    status: 'Applied',  
    appliedOn: '2026-09-16',  
    jobUrl: 'https://example.com/jobs/nodetest',  
    notes: 'Created by the Node.js API test suite.',  
    ...overrides  
  };  
}
```

What's happening:

Date.now() produces a timestamp, so the marker changes between test runs. payload() creates a standard valid application, while overrides lets individual tests change only the fields they need.

Why it matters:

Tests can safely create temporary records without confusing them with normal database data.

Byte 4: Checking the Legacy Table

Builds on: Byte 3

In plain terms:

The project has an old Applications table that must remain untouched. This helper counts its rows before and after testing.

The code:

```
async function applicationsCount() {  
  const result = await pool.query(  
    'SELECT COUNT(*)::int AS count FROM "Applications"  
  );  
  return result.rows[0].count;  
}
```

What's happening:

A PostgreSQL COUNT(*) query counts the rows in the original table. ::int converts PostgreSQL's count result into an integer.

Why it matters:

The Node.js implementation is supposed to use nodejs_applications, not modify the original Applications table.

Byte 5: Cleaning Up Test Records

Builds on: Byte 3

In plain terms:

Tests create database records, so those records must be removed afterward. The cleanup function deletes only records belonging to this test suite.

The code:

```
async function cleanup() {  
  await pool.query(  
    'DELETE FROM nodejs_applications WHERE company LIKE $1',  
    [`${MARKER}%`]  
  );  
}
```

What's happening:

The SQL uses a parameter \$1 instead of inserting the marker directly into the query. LIKE finds companies beginning with the test marker.

Why it matters:

This prevents test data from accumulating in PostgreSQL and avoids accidentally deleting unrelated applications.

Byte 6: Preparing the Test Environment

Builds on: Bytes 4–5

In plain terms:

Before any test runs, the suite makes sure the Node table exists, removes leftover test records, and remembers how many legacy records exist.

The code:

```
before(async () => {  
  await ensureSchema();  
  await cleanup();  
  legacyCountBefore = await applicationsCount();  
});
```

What's happening:

before() is a lifecycle hook that runs before the test suite. The database schema is prepared, old test records are removed, and the original table's row count is saved.

Why it matters:

Every test run starts from a controlled state instead of depending on what happened during an earlier run.

Byte 7: Protecting the Database After Testing

Builds on: Byte 6

In plain terms:

After all tests finish, the suite cleans its records and verifies that the original table still has exactly the same number of rows.

The code:

```
after(async () => {  
  await cleanup();  
  
  const legacyCountAfter = await applicationsCount();  
  
  assert.equal(  
    legacyCountAfter,  
    legacyCountBefore,
```

```
'legacy Applications table must remain untouched by the test suite'  
);
```

```
await pool.end();  
});
```

What's happening:

The cleanup runs first. Then the final legacy-table count is compared with the initial count. Finally, the PostgreSQL connection pool is closed.

Why it matters:

This gives the entire test suite a database-safety check, not just individual CRUD checks.

Byte 8: Testing GET All Applications

Builds on: Bytes 2–7

In plain terms:

This test verifies that the API can create applications and then retrieve them through GET /api/applications, with records ordered by newest ID first.

The code:

```
const first = await request(app)  
  .post('/api/applications')  
  .send(payload())  
  .expect(201);  
  
const second = await request(app)  
  .post('/api/applications')  
  .send(payload({  
    company: `${MARKER} Co 2`,  
    status: 'Interview'  
  })))  
  .expect(201);  
  
const res = await request(app)  
  .get('/api/applications')
```

```
.expect(200);
```

What's happening:

Two records are created first. Then the GET endpoint is called. The test later extracts their IDs and checks that they appear in descending order.

Why it matters:

It verifies both API retrieval and the ordering contract expected by the frontend.

Byte 9: Verifying Response Shape

Builds on: Byte 8

In plain terms:

An API response has a **contract**, meaning a defined structure that the frontend expects. This test checks that every required field exists.

The code:

```
for (const field of [  
  'id', 'company', 'role', 'status',  
  'appliedOn', 'jobUrl', 'notes', 'createdAt'  
) {  
  assert.ok(field in created.body);  
}
```

What's happening:

The test loops through the expected frontend fields and verifies that each one exists in the API response.

Why it matters:

An API could return HTTP 200 but still break the Vue frontend if a required property disappeared or was renamed.

Byte 10: Testing GET by ID

Builds on: Byte 8

In plain terms:

This checks that requesting `/api/applications/:id` returns the specific application associated with that ID.

The code:

```
const created = await request(app)  
  .post('/api/applications')
```

```
.send(payload())
```

```
.expect(201);
```

```
const res = await request(app)
```

```
.get(`/api/applications/${created.body.id}`)
```

```
.expect(200);
```

```
assert.equal(res.body.id, created.body.id);
```

```
assert.equal(res.body.company, `${MARKER} Co`);
```

What's happening:

A record is created, its generated ID is captured, and that ID is used in the GET request. The returned record is then compared with the created record.

Why it matters:

It confirms that the API correctly connects the URL ID to the corresponding PostgreSQL record.

Byte 11: Testing Missing and Malformed IDs

Builds on: Byte 10

In plain terms:

The API should not pretend that every ID is valid. These tests check both a valid-looking but nonexistent ID and completely malformed IDs.

The code:

```
await request(app)
```

```
.get('/api/applications/999999999')
```

```
.expect(404);
```

```
for (const bad of ['abc', '1.5', '--1', '12abc']) {
```

```
  await request(app)
```

```
    .get(`/api/applications/${bad}`)
```

```
    .expect(404);
```

```
}
```

What's happening:

A huge integer that doesn't exist should produce 404. Strings, decimals, and malformed values should also produce 404.

Why it matters:

This tests the API's defensive handling of invalid resource identifiers instead of allowing malformed input to reach database queries.

Byte 12: Testing POST Creation

Builds on: Bytes 8–11

In plain terms:

This test verifies the complete creation flow: send an application to the API, receive the created record, and confirm that it was actually persisted in PostgreSQL.

The code:

```
const res = await request(app)
  .post('/api/applications')
  .send(payload())
  .expect(201);

const fromDb = await pool.query(
  'SELECT * FROM nodejs_applications WHERE id = $1',
  [res.body.id]
);

assert.equal(fromDb.rowCount, 1);
```

What's happening:

POST should return 201 Created. The returned ID is then used in a direct database query to verify that the record really exists.

Why it matters:

A successful HTTP response alone doesn't prove database persistence. This test checks both API behavior and actual storage.

Byte 13: Testing Wishlist's Special Rule

Builds on: Byte 12

In plain terms:

The application rules allow a Wishlist application to exist without an application date. This test verifies that special case.

The code:

```
await request(app)
  .post('/api/applications')
  .send(payload({
    status: 'Wishlist',
    appliedOn: null
  })))
  .expect(201);
```

What's happening:

The normal payload is changed so that the status is Wishlist and appliedOn is null. The API is expected to accept it.

Why it matters:

It verifies a business rule that differs from normal applications, where appliedOn is required.

Byte 14: Testing Required-Field Validation

Builds on: Byte 12

In plain terms:

The API must reject an application when required fields are missing.

The code:

```
const res = await request(app)
  .post('/api/applications')
  .send({})
  .expect(400);

assert.ok(res.body.errors.Company);
assert.ok(res.body.errors.Role);
assert.ok(res.body.errors.Status);
```

What's happening:

An empty object is submitted. The API should respond with 400 Bad Request and provide validation errors for the missing fields.

Why it matters:

Validation prevents incomplete or invalid data from reaching PostgreSQL.

Byte 15: Testing Specific Validation Rules

Builds on: Byte 14

In plain terms:

The suite doesn't only test missing fields. It checks individual business rules such as whitespace-only company names, invalid statuses, missing dates, invalid URLs, and oversized notes.

The code:

```
await request(app)

  .post('/api/applications')

  .send(payload({ company: ' ' }))

  .expect(400);
```

```
await request(app)

  .post('/api/applications')

  .send(payload({ status: 'Hired' }))

  .expect(400);
```

```
await request(app)

  .post('/api/applications')

  .send(payload({ status: 'Offer', appliedOn: null }))

  .expect(400);
```

What's happening:

Each request intentionally violates one validation rule and expects 400.

Why it matters:

Testing individual rules makes it easier to detect exactly which validation behavior breaks after a code change.

Byte 16: Testing URL and Notes Limits

Builds on: Byte 15

In plain terms:

The API also validates optional fields. A job URL must be valid, and notes cannot exceed the defined 1000-character limit.

The code:

```
await request(app)

  .post('/api/applications')

  .send(payload({ jobUrl: 'not-a-url' }))

  .expect(400);

await request(app)

  .post('/api/applications')

  .send(payload({ notes: 'n'.repeat(1001) }))

  .expect(400);
```

What's happening:

The first request provides an invalid URL. The second creates 1001 characters, one beyond the allowed maximum.

Why it matters:

Boundary tests like this protect the API from invalid data that might otherwise reach the database or frontend.

Byte 17: Testing Malformed JSON**Builds on:** Byte 15**In plain terms:**

Not every bad request is a normal JavaScript object. The client might send invalid JSON syntax, so the API must handle that safely.

The code:

```
await request(app)

  .post('/api/applications')

  .set('Content-Type', 'application/json')

  .send('{ "company": '})

  .expect(400);
```

What's happening:

The request explicitly says it contains JSON but sends incomplete JSON syntax. Express's JSON parser should reject it and produce a 400 response.

Why it matters:

It tests an error occurring before normal application validation even begins.

Byte 18: Testing PUT Updates

Builds on: Bytes 12–17

In plain terms:

The PUT tests verify that an existing application can be completely updated and that the changes are actually stored in PostgreSQL.

The code:

```
const created = await request(app)
  .post('/api/applications')
  .send(payload())
  .expect(201);

const res = await request(app)
  .put(`/api/applications/${created.body.id}`)
  .send(updated)
  .expect(200);

assert.equal(res.body.company, `${MARKER} Updated`);
assert.equal(res.body.status, 'Offer');
```

What's happening:

A record is created first. Then PUT sends replacement data for that record. The returned response is checked to make sure the new values are present.

Why it matters:

This verifies the update part of CRUD and ensures the API returns the updated resource.

Byte 19: Verifying PUT Persistence

Builds on: Byte 18

In plain terms:

The PUT response isn't trusted by itself. The test queries PostgreSQL afterward to confirm that the update reached the database.

The code:

```
const fromDb = await pool.query(
  'SELECT * FROM nodejs_applications WHERE id = $1',
  [created.body.id]
);

assert.equal(fromDb.rows[0].company, `${MARKER} Updated`);
```

What's happening:

The updated application's ID is used to read the database directly. The stored company name must match the updated value.

Why it matters:

This catches bugs where the API returns updated data but fails to actually persist the change.

Byte 20: Testing Invalid PUT Requests

Builds on: Byte 18

In plain terms:

PUT must handle the same kinds of problems as GET and POST: nonexistent IDs, malformed IDs, and invalid application data.

The code:

```
await request(app)
  .put('/api/applications/999999999')
  .send(payload())
  .expect(404);

await request(app)
  .put('/api/applications/abc')
  .send(payload())
  .expect(404);

await request(app)
  .put(`/api/applications/${created.body.id}`)
  .send(payload({ status: 'Hired' }))
  .expect(400);
```

What's happening:

The first two requests test resource identification. The third tests validation during an update.

Why it matters:

CRUD endpoints should apply consistent validation and error handling, not only the POST endpoint.

Byte 21: Testing DELETE

Builds on: Bytes 18–20

In plain terms:

DELETE should remove an existing record and return 204 No Content. The test then confirms that the record really disappeared.

The code:

```
await request(app)
  .delete(`/api/applications/${created.body.id}`)
  .expect(204);
```

```
await request(app)
  .get(`/api/applications/${created.body.id}`)
  .expect(404);
```

What's happening:

The application is deleted first. A subsequent GET tries to retrieve the same ID and should receive 404.

Why it matters:

This verifies both the HTTP contract and the practical effect of deletion.

Byte 22: Verifying DELETE at the Database Level

Builds on: Byte 21

In plain terms:

The test also directly checks PostgreSQL to ensure the deleted row is really gone.

The code:

```
const fromDb = await pool.query(
  'SELECT * FROM nodejs_applications WHERE id = $1',
  [created.body.id]
);
```

```
assert.equal(fromDb.rowCount, 0);
```

What's happening:

The database query searches for the deleted ID. Zero matching rows confirms the deletion reached PostgreSQL.

Why it matters:

It prevents a false-positive test where the API claims deletion but the database still contains the record.

Byte 23: Testing DELETE Error Cases

Builds on: Byte 21

In plain terms:

DELETE should also reject IDs that don't identify an existing application.

The code:

```
await request(app)
  .delete('/api/applications/999999999')
  .expect(404);
```

```
await request(app)
  .delete('/api/applications/abc')
  .expect(404);
```

What's happening:

One request uses a nonexistent numeric ID and another uses a malformed ID. Both should return 404.

Why it matters:

It keeps resource lookup behavior consistent across GET, PUT, and DELETE.

Byte 24: Proving the Legacy Table Is Untouched

Builds on: Bytes 4, 7, and 21

In plain terms:

The final database test verifies the central migration requirement: Node.js CRUD operations must write only to `nodejs_applications`, never to the original `Applications` table.

The code:

```
const before = await applicationsCount();
```

```
const created = await request(app)
```

```
.post('/api/applications')
```

```
.send(payload())
```

```
.expect(201);
```

```
await request(app)
```

```
.put(`/api/applications/${created.body.id}`)
```

```
.send(payload())
```

```
.expect(200);
```

```
await request(app)
```

```
.delete(`/api/applications/${created.body.id}`)
```

```
.expect(204);
```

```
const after = await applicationsCount();
```

```
assert.equal(after, before);
```

What's happening:

The test performs the complete create → update → delete cycle and compares the legacy table's row count before and after.

Why it matters:

This directly protects the separation between the old .NET data and the new Node.js implementation.

PUTTING IT TOGETHER

This file tests the Node.js API from the outside using **Supertest**, while also checking PostgreSQL directly when persistence matters. The suite first prepares the database and creates isolated test data using the NodeTest marker. It then covers the complete CRUD lifecycle: GET, POST, PUT, and DELETE, along with validation and malformed-input cases. Finally, it verifies that all Node.js operations use `nodejs_applications` and leave the legacy Applications table unchanged. After cleanup, the PostgreSQL connection pool is closed so the test process can exit cleanly.

validation.test.js

Byte 1: Purpose of This Test File

Builds on: None — starting point

In plain terms:

This file contains **unit tests for the backend validation logic**. Unlike the API tests, these tests do not need PostgreSQL or HTTP requests—they directly call `validateApplication()` and check whether different inputs are accepted or rejected.

The code:

```
const { describe, it } = require('node:test');
const assert = require('node:assert/strict');

const {
  validateApplication,
  ALLOWED_STATUSES
} = require('../src/validation');
```

What's happening:

Node's test framework provides `describe()` and `it()`. `assert` checks expected results. The actual validation function and its list of allowed statuses are imported directly.

Why it matters:

Testing validation separately makes failures easier to locate. If a validation rule breaks, we know the problem is in validation rather than Express, PostgreSQL, or the API routing.

Byte 2: Creating a Reusable Valid Input

Builds on: Byte 1

In plain terms:

Most validation tests need a valid application as their starting point. `validInput()` creates that valid baseline, while `overrides` lets each test change one field.

The code:

```
function validInput(overrides = {}) {
  return {
    company: 'Acme Labs',
    role: 'Backend Intern',
    status: 'Applied',
```

```
    appliedOn: '2026-09-16',  
    jobUrl: 'https://example.com/jobs/acme',  
    notes: 'Sample record.',  
    ...overrides  
  };  
}
```

What's happening:

The function returns a complete valid application. If a test passes `{ status: 'Hired' }`, the spread operator replaces only the status value.

Why it matters:

Tests stay short and focused. Each test can change only the value relevant to the rule being tested.

Byte 3: Verifying the Supported Statuses

Builds on: Byte 2

In plain terms:

An application can have only five supported statuses. The first tests make sure that the validation module exposes exactly those statuses.

The code:

```
assert.deepEqual(  
  ALLOWED_STATUSES,  
  ['Wishlist', 'Applied', 'Interview', 'Offer', 'Rejected']  
);
```

What's happening:

`deepEqual()` compares the contents of the two arrays, not just whether they are the same object.

Why it matters:

This protects the application's status contract. Accidentally adding, removing, or changing a status would cause this test to fail.

Byte 4: Testing Every Valid Status

Builds on: Byte 3

In plain terms:

Knowing the allowed list exists isn't enough. The test also checks that every individual status is actually accepted by the validator.

The code:

```
for (const status of ALLOWED_STATUSES) {  
  const appliedOn =  
    status === 'Wishlist' ? null : '2026-09-16';  
  
  const { errors } =  
    validateApplication(validInput({ status, appliedOn }));  
  
  assert.equal(errors, null);  
}
```

What's happening:

The test loops through all five statuses. Wishlist receives no application date because that status has a special rule; the other statuses receive a date.

Why it matters:

It verifies the complete valid-status path instead of testing only one example such as Applied.

Byte 5: Rejecting Unknown and Missing Statuses

Builds on: Byte 4

In plain terms:

The validator should reject both an unsupported status such as Hired and a completely missing status.

The code:

```
const { errors, value } =  
  validateApplication(validInput({ status: 'Hired' }));  
  
assert.equal(value, null);  
assert.ok(  
  errors.Status.join(' ')  
    .includes('Wishlist, Applied, Interview, Offer, Rejected')  
);
```

What's happening:

An unknown status produces validation errors and no normalized value. A separate test checks that undefined produces a "Status is required." error.

Why it matters:

This prevents invalid status values from reaching the database and ensures missing required data is reported clearly.

Byte 6: Testing Company and Role Requirements

Builds on: Byte 2

In plain terms:

company and role must contain meaningful text between 2 and 100 characters. These tests cover valid values, missing values, whitespace, and length boundaries.

The code:

```
const { errors } =  
  validateApplication(  
    validInput({ company: "", role: "" })  
  );
```

```
assert.ok(errors.Company);
```

```
assert.ok(errors.Role);
```

What's happening:

An empty company and role should generate errors. Other tests check whitespace-only values, one-character values, and strings longer than 100 characters.

Why it matters:

These tests enforce the data-quality rules before information reaches PostgreSQL.

Byte 7: Testing Input Normalization

Builds on: Byte 6

In plain terms:

Validation doesn't only reject bad data—it also cleans acceptable data. Here, leading and trailing spaces are removed from company and role.

The code:

```
const { value } = validateApplication(  
  validInput({  
    company: ' Acme ',  
    role: ' Dev '  
  })  
)
```

```
);
```

```
assert.equal(value.company, 'Acme');
```

```
assert.equal(value.role, 'Dev');
```

What's happening:

The input contains unnecessary spaces, but the normalized output contains clean values.

Why it matters:

Normalization keeps stored data consistent. " Acme " and "Acme" don't become two visually different versions of the same company.

Byte 8: Testing the AppliedOn/Wishlist Rule

Builds on: Bytes 4–5

In plain terms:

appliedOn follows a business rule: it is required for normal application statuses but optional for Wishlist.

The code:

```
for (const status of [
  'Applied',
  'Interview',
  'Offer',
  'Rejected'
]) {
  const { errors } =
    validateApplication(
      validInput({ status, appliedOn: null })
    );

  assert.ok(errors.AppliedOn);
}
```

What's happening:

Each non-Wishlist status is tested with a missing application date. The validator should reject every one.

Why it matters:

This protects an important relationship between two fields instead of validating each field independently.

Byte 9: Allowing Wishlist Without a Date

Builds on: Byte 8

In plain terms:

The exception to the previous rule is Wishlist. A company being saved for future consideration doesn't need an application date.

The code:

```
const { errors, value } =
  validateApplication(
    validInput({
      status: 'Wishlist',
      appliedOn: null
    })
  );

assert.equal(errors, null);
assert.equal(value.appliedOn, null);
```

What's happening:

The validator accepts the missing date and preserves it as null.

Why it matters:

This confirms the special business rule works rather than accidentally making appliedOn mandatory for every status.

Byte 10: Testing Date Validation

Builds on: Bytes 8–9

In plain terms:

A date must not only exist—it must actually be a valid calendar date. These tests check both an obviously invalid string and an impossible date.

The code:

```
const { errors } =
  validateApplication(
```

```
    validInput({ appliedOn: 'not-a-date' })  
  );
```

```
assert.ok(errors.AppliedOn);
```

And:

```
const { errors } =  
  validateApplication(  
    validInput({ appliedOn: '2026-02-30' })  
  );
```

```
assert.ok(errors.AppliedOn);
```

What's happening:

The first value isn't a date at all. The second looks like a date but February 30 does not exist.

Why it matters:

It prevents invalid calendar values from being stored as application dates.

Byte 11: Testing Optional Job URL and Notes

Builds on: Byte 2

In plain terms:

jobUrl and notes are optional fields. Missing values should therefore be accepted and represented as null.

The code:

```
const { errors, value } =  
  validateApplication(  
    validInput({  
      jobUrl: null,  
      notes: null  
    })  
  );
```

```
assert.equal(errors, null);
```

```
assert.equal(value.jobUrl, null);
```

```
assert.equal(value.notes, null);
```

What's happening:

Both optional fields are explicitly set to null. Validation succeeds and the normalized result keeps them as null.

Why it matters:

It distinguishes optional information from invalid information. An absent optional field should not automatically cause a validation failure.

Byte 12: Normalizing Empty Optional Fields

Builds on: Byte 11

In plain terms:

Empty strings and whitespace-only values for optional fields are treated the same as missing values.

The code:

```
const { errors, value } =  
  validateApplication(  
    validInput({  
      jobUrl: ' ',  
      notes: "  
    })  
  );
```

```
assert.equal(errors, null);  
assert.equal(value.jobUrl, null);  
assert.equal(value.notes, null);
```

What's happening:

Instead of storing meaningless empty strings, the validator converts them into null.

Why it matters:

This keeps optional database fields clean and gives the application one consistent representation for “no value.”

Byte 13: Testing Job URL Rules

Builds on: Byte 11

In plain terms:

If a job URL is provided, it must be a valid URL and use HTTP or HTTPS.

The code:

```
const { errors } =  
  validateApplication(  
    validInput({ jobUrl: 'not-a-url' })  
  );
```

```
assert.ok(errors.JobUrl);
```

And:

```
const { errors } =  
  validateApplication(  
    validInput({  
      jobUrl: 'ftp://example.com/job'  
    })  
  );
```

```
assert.ok(errors.JobUrl);
```

What's happening:

The first value isn't a valid URL. The second is a URL but uses ftp, which is outside the application's allowed HTTP/HTTPS rule.

Why it matters:

It prevents invalid or unsupported links from being stored as job URLs.

Byte 14: Testing the Notes Length Boundary

Builds on: Byte 11

In plain terms:

Notes have a maximum length of 1000 characters. The tests check both sides of that boundary.

The code:

```
const { errors } =  
  validateApplication(  
    validInput({
```

```
validInput({
  notes: 'n'.repeat(1001)
})
);
```

```
assert.ok(errors.Notes);
```

And:

```
const { errors } =
  validateApplication(
    validInput({
      notes: 'n'.repeat(1000)
    })
  );
```

```
assert.equal(errors, null);
```

What's happening:

1001 characters must fail, while exactly 1000 characters must pass.

Why it matters:

Boundary testing catches off-by-one errors such as accidentally allowing 1001 characters or rejecting a valid 1000-character value.

Byte 15: Rejecting Invalid Request Bodies

Builds on: Bytes 2–14

In plain terms:

The validator expects an application represented as an object. It should reject completely different types such as strings, numbers, arrays, null, or undefined.

The code:

```
for (const body of [
  null,
  undefined,
  'text',
  42,
```

```
    []  
  }) {  
    const { errors, value } =  
      validateApplication(body);  
  
    assert.ok(errors);  
    assert.equal(value, null);  
  }  
}
```

What's happening:

Each invalid body is passed directly to the validator. Every one must produce errors and no normalized value.

Why it matters:

This protects the validation layer from unexpected input before it tries to access fields such as company or status.

PUTTING IT TOGETHER

This file tests the validation layer directly, without Express, HTTP, or PostgreSQL. It starts with a reusable valid application and then systematically changes one rule at a time. The tests cover allowed statuses, company/role requirements, the special Wishlist date rule, dates, URLs, notes, normalization, and completely invalid request bodies. Together, they define the expected input contract for `validateApplication()`. If the validation implementation changes and violates any of these rules, the corresponding unit test should fail immediately.